Sky130 FP16 Calculator

ASIC User Manual & Technical Reference

Half-Precision Floating-Point Arithmetic Core

Operations: ADD · SUB · MUL · DIV · SQRT · POW

Interface: UART Serial Communication

Julian Pacheco - Alejandro Olarte | Technology:

SKY130 | Version: 1.0 TinyTapeout

Submission — 1x1 Tile

Table of Contents

1. Overview

The Sky130 FP16 Calculator is a custom ASIC designed and fabricated on the open-source SKY130 process node through the TinyTapeout program. It implements a half-precision floating-point arithmetic unit (FPU) capable of performing six fundamental mathematical operations on 16-bit IEEE 754 numbers.

Communication with the chip is established via a standard UART serial interface, making it easy to integrate with any microcontroller, FPGA, or PC. The design targets a clock frequency of 10 MHz and occupies a single 1x1 TinyTapeout tile.

Feature	Value
Technology	SKY130 (130nm CMOS)
Number Format	FP16 — IEEE 754 Half Precision (16-bit)
Operations	ADD, SUB, MUL, DIV, SQRT, POW
Interface	UART 115200 baud, 8N1
Clock Frequency	10 MHz (nominal)
Tile Size	1x1 TinyTapeout tile (~160 x 225 µm)
Supply Voltage	1.8 V (core) / 3.3 V (IO)

2. System Architecture

The chip consists of three main blocks connected internally:

Block	Description
UART RX	Receives serial packets from the host PC or microcontroller
FP16 Calculator Core	Performs the arithmetic operation using FloPoCo-generated operators
UART TX	Sends the result and status flags back to the host

The data flow is strictly sequential: the host sends a command packet, the calculator processes it, and returns a response packet. No simultaneous operations are supported.

The FloPoCo library was used to generate optimized VHDL operators for FP16 arithmetic, which were then synthesized to Verilog and integrated with the UART controller.

3. Pin Description

The chip uses the standard TinyTapeout pinout with 8 input pins (ui), 8 output pins (uo), and 8 bidirectional pins (uio). Only two pins are actively used:

Input Pins (ui)

Pin	Name	Direction	Description
ui[0]	UART_RX	Input	Serial data input — connect to TX of host device
ui[1]	—	Input	Unused — leave floating or connect to GND
ui[2]	—	Input	Unused — leave floating or connect to GND
ui[3]	—	Input	Unused — leave floating or connect to GND
ui[4]	—	Input	Unused — leave floating or connect to GND
ui[5]	—	Input	Unused — leave floating or connect to GND
ui[6]	—	Input	Unused — leave floating or connect to GND
ui[7]	—	Input	Unused — leave floating or connect to GND

Output Pins (uo)

Pin	Name	Direction	Description
uo[0]	UART_TX	Output	Serial data output — connect to RX of host device
uo[1]	—	Output	Unused — do not connect
uo[2]	—	Output	Unused — do not connect
uo[3]	—	Output	Unused — do not connect
uo[4]	—	Output	Unused — do not connect
uo[5]	—	Output	Unused — do not connect
uo[6]	—	Output	Unused — do not connect
uo[7]	—	Output	Unused — do not connect

Power and Control Pins

Pin	Description
clk	Main clock input — 10 MHz recommended
rst_n	Active-low reset — pull HIGH for normal operation

ena	Enable signal — pull HIGH to enable the chip
VPWR	Core power supply — 1.8 V
VGND	Ground reference — 0 V

4. UART Communication Protocol

The chip uses a simple fixed-length packet protocol over UART at 115200 baud, 8 data bits, no parity, 1 stop bit (8N1).

Command Packet (Host → Chip)

Total length: 7 bytes

Byte	Value	Description
0	0xAA	Header — start of packet
1	0x0X	Operation code (lower 3 bits): 0=ADD, 1=SUB, 2=MUL, 3=DIV, 4=SQRT, 5=POW
2	A[15:8]	Operand A — high byte (FP16 big-endian)
3	A[7:0]	Operand A — low byte
4	B[15:8]	Operand B — high byte (FP16 big-endian)
5	B[7:0]	Operand B — low byte
6	0x55	Footer — end of packet

Response Packet (Chip → Host)

Total length: 5 bytes

Byte	Value	Description
0	0xBB	Header — start of response
1	R[15:8]	Result — high byte (FP16 big-endian)
2	R[7:0]	Result — low byte
3	FLAGS	Status flags byte (see Section 9)
4	0x66	Footer — end of response

Note: For SQRT and ABS operations, operand B is ignored and should be sent as 0x0000.

Note: For POW (A^N), operand B encodes the integer exponent N in bits [4:0] (maximum N = 31). Send B as a raw integer, not as FP16.

5. Supported Operations

Code	OP	Formula	Operand A	Operand B	Notes
0x0	ADD	A + B	FP16 number	FP16 number	Full precision
0x1	SUB	A - B	FP16 number	FP16 number	Full precision
0x2	MUL	A × B	FP16 number	FP16 number	Full precision
0x3	DIV	A / B	FP16 number	FP16 number	B=0 returns ±Inf
0x4	SQRT	sqrt(A)	FP16 number	Ignored (0)	A<0 returns NaN
0x5	POW	A ^ N	FP16 base	Integer N [4:0]	Max N=31

6. Number Format — FP16 IEEE 754

The chip uses the IEEE 754 half-precision floating-point format (binary16). Each number is 16 bits wide:

Bits	Field	Size	Description
[15]	Sign	1 bit	0 = positive, 1 = negative
[14:10]	Exponent	5 bits	Biased exponent (bias = 15)
[9:0]	Mantissa	10 bits	Fractional part (implicit leading 1)

Property	Value
Smallest positive normal	~6.10 × 10^-5
Largest positive normal	65504
Precision	~3 decimal digits
Special values	0, -0, +Inf, -Inf, NaN

To convert a Python float to FP16 bytes for transmission:

```
import struct
fp16 = struct.pack('>e', 3.14) # 2 bytes, big-endian FP16
```

7. Quick Start Guide

Step 1 — Hardware Connection

Connect ui[0] (UART_RX) to the TX pin of your USB-to-UART adapter. Connect uo[0] (UART_TX) to the RX pin of your adapter. Connect GND, VPWR (1.8V), and apply clock at 10 MHz. Pull rst_n and ena HIGH.

Step 2 — Configure Serial Port

Open your serial terminal or script at 115200 baud, 8N1, no flow control. Common USB-to-UART adapters: CP2102, CH340, FT232.

Step 3 — Send a Command

Build a 7-byte packet: [0xAA, OP, A_hi, A_lo, B_hi, B_lo, 0x55] and send it.

Step 4 — Read the Response

Wait for 5 bytes: [0xBB, R_hi, R_lo, FLAGS, 0x66]. Decode the result using IEEE 754 FP16 format.

8. Example: Python Host Script

The following Python script demonstrates how to send an ADD operation (3.14 + 2.71) to the chip and read the result:

```
import serial, struct

# FP16 conversion helpers
def to_fp16(f):
return struct.pack('>e', f)

def from_fp16(b):
return struct.unpack('>e', b)[0]

# Open serial port
ser = serial.Serial('COM3', 115200, timeout=2) # adjust port

# Build command: ADD (op=0), A=3.14, B=2.71
op = 0x00 # ADD
A = to_fp16(3.14)
B = to_fp16(2.71)
pkt = bytes([0xAA, op]) + A + B + bytes([0x55])

# Send and receive
ser.write(pkt)
resp = ser.read(5)

# Decode response
if resp[0] == 0xBB and resp[4] == 0x66:
result = from_fp16(resp[1:3])
flags = resp[3]
print(f'Result: {result:.4f}') # Expected: ~5.8516
print(f'NaN={bool(flags&0x80)} OVF={bool(flags&0x40)}')
```

9. Error Flags and Status Codes

The FLAGS byte in the response packet encodes the following status bits:

Bit	Mask	Name	Description
7	0x80	NAN	Result is Not-a-Number (e.g. sqrt of negative)
6	0x40	OVERFLOW	Result is +Infinity or -Infinity
5	0x20	ZERO	Result is zero
4	0x10	CMP_EQ	Comparison: A equals B
3	0x08	CMP_GT	Comparison: A greater than B
2	0x04	CMP_LT	Comparison: A less than B
1	0x02	—	Reserved — always 0
0	0x01	—	Reserved — always 0

10. Electrical Characteristics

Parameter	Min	Typical	Max	Unit
Core supply (VPWR)	1.7	1.8	1.9	V
IO supply	3.0	3.3	3.6	V
Clock frequency	1	10	25	MHz
UART baud rate	—	115200	—	baud
Operating temp.	-40	25	85	°C

11. Frequently Asked Questions

Q: Can I use 3.3V logic for UART?

A: Yes. The IO pads of the SKY130 process are 3.3V tolerant. A standard USB-to-UART adapter with 3.3V logic levels works directly.

Q: What happens if I divide by zero?

A: The result will be +Infinity or -Infinity depending on the sign of operand A, and the OVERFLOW flag will be set in the response.

Q: Why does SQRT of a negative number return NaN?

A: Square root of a negative real number is not defined. The chip returns the IEEE 754 canonical NaN value (0x7E00) and sets the NAN flag.

Q: What is the maximum exponent for POW?

A: The POW operation accepts integer exponents from 0 to 31. The exponent is encoded in the lower 5 bits of operand B as a raw integer.

Q: Can I chain multiple operations?

A: Yes. Simply send successive packets. Wait for the response of each operation before sending the next command.

Q: What precision can I expect?

A: FP16 provides approximately 3 significant decimal digits. For higher precision applications, consider using the result as an intermediate value only.